# Weiterarbeit für GPT-6 im Chat-/Codex-Modus oder Claude Fable5.1

Stand: Sol-Fortsetzung nach Fable-Genesis,01.11.2026. Repository Ullmann27/device-ai, Draft-PR#335, Branch research/continuum-sol61-post-fable-20261101, Zielbranch weiterhin Fable research/continuum-fable51-genesis-20261101. Offizielles privates Repository, kein gesperrter Code und kein nur lokaler Patch. Alle Dateien normal editierbar; keine Verschlüsselung/Schreibsperre neu eingeführt.

## Direkt einsetzbarer Auftrag

Du setzt die Arbeit von Fable5.1 und Sol6.1 fort. Starte NICHT erneut mit CDX1-Training, einer neuen Gedächtnisdatenbank, allgemeinen Architekturberichten oder einem Main-Merge. Hole den echten Remote-Stand, lies AGENTS, SOL61-POST-FABLE-HANDOFF und diese Datei. Danach FABLE51-GENESIS-HANDOFF, CONTINUATION und CDX1-ORACLE-BOUND nur für die tatsächlich bearbeitete Funktion heranziehen.

Fable-Übergabe-HEAD702b57ae ist vollständig enthalten. Sols letzter funktionaler Codecommit ist 6e1825b09ffca1c811414f9eeb7790607ef4c448. Ein nachfolgender Dokumentationscommit ist ein anderer HEAD. Prüfe PR335 und den unabhängigen Render-Proof: Ein älteres PASS darf niemals für einen neueren SHA verwendet werden. Der abschließende Proof/Deploy ist im PR verlinkt; diese statische Datei kann ihren eigenen zukünftigen Commit nicht kennen.

1. git fetch origin; tatsächliche HEADs und git status --short prüfen. PR335/334 und parallele Änderungen abgleichen. Eigenen isolierten Folgebranch vom geprüften PR335-HEAD erstellen, Draft gegen diesen Integrationsstand. Keine fremde Branchspitze zurücksetzen.
2. Erster technischer Check: finalen Android-Proof lesen. QA-Service srv-dbjktl5g1s2s7381v030, Workspace tea-d7qe6dosfn5c73e2rnag, AutoDeploy aus. https://continuum-sol61-post-fable-qa-20261101.onrender.com/ARCHITECTURE-PROOF.json und STATUS.txt abrufen, sourceCommit/schema3/runId/Tests/unsignedArtifactHashes/BinaryIdentity prüfen. Bei ausstehender/fehlgeschlagener letzter QA zuerst den konkreten Fehler beheben. Bestehendes scripts/render-continuum-m8-qa.sh verwenden, keine zweite Pipeline.
3. Nach grünem Android-Stand zuerst Fold7-Abnahmematrix ausführen oder vorbereiten. DEVICE_NOT_RUN bleibt bis zum realen Test. Neues unsigniertes0.3.35/code36 ist KEIN Besitzerupdate. Bekannter publizierter Owner-Kanalcode36 aus8417 ist unabhängig vom Sol-Code. Vor erneut installierbarem Update installierten Code und Originalsignatur real prüfen, monotone Versions-/Releasefreigabe separat einholen. Kein Server-/Pin-/Signierschlüsselwechsel.
4. Erster neuer Kotlin-Patch: S6 RIS-Artikel/Paginierung. Bearbeite bestehenden LegalRisDiscoveryProvider und dessen Tests. Sichere zuerst fehlschlagende Art7B-VG-Regressionsfälle mit docs/evidence/sol61-post-fable-20261101/s6-preflight/. Die tatsächlichen offiziellen Antworten zeigen Artikelnummer und ArtikelParagraphAnlage, bei Artikeln heißt Dokumenttyp trotzdem Paragraph. Strukturierte offizielle Titel+Abschnitt-Filter liefern12Art7-Treffer, Seite1nur10; hole Seite2 über denselben dokumentierten Endpunkt, prüfe das echte Antwortformat und bewahre historische/Gültigkeitsdaten. Keine Gesetzesnummern oder API-Felder raten. Durchgehende checkAccess-Prüfungen, Gesamtbudget, feste Maximalzahl Seiten, Revisions-/Duplikatprüfung und strikte Normzuordnung testen. Nicht nachträglich Fixture-Labels an schwachen Code anpassen.
5. Danach S4 Rückkopplung oder S3 weiterer enger Attributtyp: vorhandenen Ledger/OwnerAttributeEvidenceScope/EWM-Mechanismus verwenden. Sols Altersantwort ist exakt, aber nur eine bestätigte Angabe zum damaligen Zeitpunkt; keine automatische Altershochrechnung oder allgemeines Personenverständnis. Neuere Negationen/Widerrufe dürfen ältere Angaben nicht wiederbeleben. Agentengespräche/gespeicherte Modellantworten/Webdaten dürfen nie zur Owner-Bestätigung werden. Vorhandene Trace-Rückmeldung ist Metadatenbewertung, kein automatisch sicheres Training. ResearchFeedback ist weiterhin isolierte synthetische Forschung ohne produktiven Caller.
6. S5 separat und isoliert: CDX2 mit Rollen/Positionen+Copy/Pointer, unabhängiger Verifier, Enthaltung. Blind-C NOT_PREPARED: Kein Korpus/Seed/Hash/Checkpoint oder Modellresultat aus dieser Sitzung vorhanden. Erst durch unabhängige Instanz disjunkten frischen Test vor Kandidatenfreeze erstellen; A/B und FableOracle sind verbrauchte Forschung, unverändert lassen. Getrennte RAW/verified/rule/CDX1Resultate, exakte Quelle/Span,

>=16/20 und5/5 sind Ziele, keine erreichten Werte. Kein produktiver Chat-/Gewichtspromotion ohne neues Gate. 7. S7/S8 erst anhand echter Aufrufer: TemporalRoleEvidenceLedger/ConstrainedResourceFabric sind noch ohne bestätigten Produktionscaller. Keine gleichnamige Fabric-Klasse erfinden, kein virtuelles RAM als physisch melden. Für Worker .github/continuum/repair-loop.mjs, repair-publish, vorhandene feste PROFILES/CHAINS/Watchdogs/TypedCodingBridge verwenden; keine aus Modell-/Webtext abgeleitete Shell und keine kostenpflichtige KI-API-Pflicht. Android-first, PC optional. Erst danach Avatar/UI.

## Reproduzierbare Tests und ehrliche Grenzen

- Vollständiger Host: bash scripts/android-kotlin-unit-check.sh <fresh-output> mit JDK21/javac und kotlinc2.0.21, offiziellen android-all15/OkHttp4.12/Okio3.6/Shizuku13.1.5Jars über CONTINUUM_ANDROID_DEPS, JUnit4.13.2/Hamcrest1.3 via CONTINUUM_JUNIT_JARS, Kotlinstdlib2.0.21 via CONTINUUM_KOTLIN_STDLIB. Script dokumentiert Details. HOST verwendet vorhandenen LiteRT-Compile-Stub; kein APK-/Devicebeleg. Für Teilkompilierung internals -module-name app behalten, sonst clearPending$app-Linkagefehler.
- Vollständige Node-/Backend-/Android-Kommandos stehen unverändert in scripts/render-continuum-m8-qa.sh. Backend npmci ohne Lifecycle-Scripts, npm run check und64Tests; QA-Node115Tests. Nicht nur die neun Proof-Parser-Tests als gesamte Node-Abnahme bezeichnen.
- Reale Gradle-/LiteRT0.18.0-Abnahme lief auf Render. Die Google-Abhängigkeit ist erreichbar; keine Versionen/Repositories/Quellen ausschließen, um eine Stand-in-Abnahme zu erhalten. Gradlecompile/JUnit/lint und unsignedAPK+AAB getrennt von Signierung/Device.
- git diff --check; gezielte RED/GREEN-Regression, vollständiger Host, Node soweit betroffen, danach commit/push und exakte Render-QA. Kandidaten mit konkretem Fehlschlag reparieren; negative Messungen erhalten.

## Abschluss jedes Folgecheckpoints

Kleine Commits und sofortiger Push, präzise PR-/Handoff-Aktualisierung mit SHA, SOURCE_INTEGRATED/HOST_VERIFIED/DEVICE_NOT_RUN-Status, Testausgaben, fehlenden Gates und erstem Anschluss. Keine Main-/Releasefreigabe automatisch ableiten. Sichere fertigen Code früh, damit eine Nutzungsunterbrechung keinen lokalen Einzelstand hinterlässt.